

4. Props: передача данных между компонентами

Цель занятия

На прошлых занятиях мы уже разбили интерфейс на компоненты и добавили интерактивность через `useState`. Теперь нужно понять, как компоненты обмениваются данными.

Главная тема занятия - `props`.

`Props` позволяют родительскому компоненту передать информацию дочернему компоненту.

Коротко:

text

Родительский компонент передает `props`.

Дочерний компонент получает `props`.

Дочерний компонент использует `props` для отображения интерфейса.

После занятия студент должен уметь:

- объяснить, что такое `props`;
- передавать строку, число, переменную и функцию через `props`;
- читать `props` внутри компонента;
- использовать деструктуризацию параметров;
- понимать, что `props` нельзя изменять внутри дочернего компонента;
- отличать `props` от `state`;
- понимать, зачем `props` делают компоненты гибкими.

1. Что говорит документация React

В актуальной документации React `props` описываются как способ коммуникации между компонентами. Родительский компонент может передать дочернему компоненту информацию через атрибуты JSX.

Пример:

javascript

```
function Profile() {  
  return <Avatar name="Анна" age={19} />  
}
```

Компонент `Avatar` получает данные:

javascript

```
function Avatar({ name, age }) {  
  return (  
    <div>  
      <p>{name}</p>  
      <p>Возраст: {age}</p>  
    </div>  
  )  
}
```

Здесь:

- `Profile` - родительский компонент;
- `Avatar` - дочерний компонент;
- `name` и `age` - `props`.

2. Props простыми словами

Props похожи на аргументы функции.

Обычная JavaScript-функция:

```
javascript
function greet(name) {
  return `Привет, ${name}!`
}

greet('Анна')
```

React-компонент:

```
javascript
function Greeting({ name }) {
  return <h1>Привет, {name}!</h1>
}

<Greeting name="Анна" />
```

В обоих случаях мы передаем значение внутрь функции. Только в React это значение приходит через props.

3. Почему props нужны

Без props компоненты часто становятся слишком жесткими.

Пример жесткого компонента:

```
javascript
function Button() {
  return <button>Reset</button>
}
```

Такой компонент всегда показывает только `Reset`. Если нужна кнопка `Toggle hint`, придется создавать еще один компонент или переписывать старый.

Гибкий компонент:

```
javascript
function Button({ label }) {
  return <button>{label}</button>
}
```

Теперь можно использовать один компонент по-разному:

```
javascript
<Button label="Reset" />
<Button label="Toggle hint" />
<Button label="Save" />
```

Один компонент, разные данные.

4. Как передать строку через props

Строку можно передать почти как обычный HTML-атрибут:

```
javascript
  <PageTitle title="Get started" />
```

Дочерний компонент получает значение:

```
javascript
  function PageTitle({ title }) {
    return <h1>{title}</h1>
  }
```

Когда React отображает компонент, он подставляет значение:

```
text
  Get started
```

5. Как передать число через props

Числа передаются через фигурные скобки:

```
javascript
  <Avatar size={100} />
```

Почему не так?

```
javascript
  <Avatar size="100" />
```

Потому что "100" - это строка, а {100} - это число.

Правило:

```
text
  Строки можно передавать в кавычках.
  JavaScript-значения передаются через фигурные скобки.
```

6. Как передать переменную через props

Если значение уже хранится в переменной, используем фигурные скобки.

```
javascript
  const title = 'Get started'

  <PageTitle title={title} />
```

Дочерний компонент:

```
javascript
  function PageTitle({ title }) {
    return <h1>{title}</h1>
  }
```

Фигурные скобки говорят React: возьми значение JavaScript-переменной.

7. Как передать функцию через props

Функции тоже можно передавать через props. Это уже есть в нашем проекте.

Компонент CounterButton:

```
javascript
function CounterButton({ count, onClick }) {
  return (
    <button type="button" className="counter" onClick={onClick}>
      Count is {count}
    </button>
  )
}
```

Использование:

```
javascript
<CounterButton count={count} onClick={handleCounterClick} />
```

Здесь:

- count - число;
- onClick - функция;
- CounterButton не знает, как именно меняется счетчик;
- CounterButton только вызывает переданную функцию при клике.

Это хороший подход: дочерний компонент отвечает за внешний вид кнопки, а родительский компонент хранит состояние и логику.

8. Как компонент получает props

React передает props в компонент как объект.

Можно написать так:

```
javascript
function PageTitle(props) {
  return <h1>{props.title}</h1>
}
```

Если компонент вызвали так:

```
javascript
<PageTitle title="Get started" />
```

то внутри props будет примерно такой объект:

```
javascript
{
  title: 'Get started'
}
```

Чаще используют деструктуризацию:

```
javascript
function PageTitle({ title }) {
  return <h1>{title}</h1>
}
```

Эта запись короче и удобнее.

9. Несколько props сразу

Компонент может получать несколько props.

```
javascript
function PageTitle({ title, fileName, toolName }) {
  return (
    <div>
      <h1>{title}</h1>
      <p>
        Edit <code>{fileName}</code> and save to test <code>{toolName}</code>
      </p>
    </div>
  )
}
```

Использование:

```
javascript
<PageTitle
  title="Get started"
  fileName="src/App.jsx"
  toolName="HMR"
/>
```

Теперь компонент `PageTitle` не привязан к одному тексту. Родитель может передать другие значения.

10. Props только для чтения

Props нельзя изменять внутри дочернего компонента.

Неправильно:

```
javascript
function PageTitle({ title }) {
  title = 'Новый заголовок'
  return <h1>{title}</h1>
}
```

Правильно: если нужно изменить данные, это должен сделать родитель через state.

```
javascript
function App() {
  const [title, setTitle] = useState('Get started')

  return <PageTitle title={title} />
}
```

Главная мысль:

```
text
Props приходят сверху вниз.
Дочерний компонент не изменяет props.
Если данные должны меняться, используем state у родителя.
```

11. Props и state: в чем разница

Props и state часто путают.

Понятие	Где находится	Кто изменяет	Для чего нужно
props	приходят от родителя	дочерний компонент не изменяет	передать данные в компонент
state	создается внутри компонента	компонент через set-функцию	хранить данные, которые меняются

Пример:

javascript

```
function CounterButton({ count, onClick }) {
  return (
    <button onClick={onClick}>
      Count is {count}
    </button>
  )
}
```

CounterButton получает count через props.

А состояние хранится в App:

javascript

```
function App() {
  const [count, setCount] = useState(0)

  function handleCounterClick() {
    setCount((count) => count + 1)
  }

  return <CounterButton count={count} onClick={handleCounterClick} />
}
```

App владеет состоянием. CounterButton только показывает данные и сообщает о клике.

12. Данные идут сверху вниз

В React данные обычно идут от родителя к детям.

text

```
App
└─ CounterButton
```

App передает:

javascript

```
<CounterButton count={count} onClick={handleCounterClick} />
```

CounterButton получает:

javascript

```
function CounterButton({ count, onClick }) {
  return <button onClick={onClick}>Count is {count}</button>
}
```

Если счетчик изменился, App заново отображается и передает новое значение count в CounterButton.

13. Props делают компоненты переиспользуемыми

Сравним два варианта.

Компонент без props:

```
javascript
function ResetButton() {
  return <button>Reset</button>
}
```

Компонент с props:

```
javascript
function ActionButton({ label, onClick }) {
  return (
    <button type="button" className="secondary-button" onClick={onClick}>
      {label}
    </button>
  )
}
```

Теперь его можно использовать для разных действий:

```
javascript
<ActionButton label="Reset" onClick={handleResetClick} />
<ActionButton label="Toggle hint" onClick={handleHintClick} />
```

Такой компонент гибче.

14. Значение по умолчанию

Иногда проп может иметь значение по умолчанию.

```
javascript
function ActionButton({ label = 'Button', onClick }) {
  return (
    <button type="button" onClick={onClick}>
      {label}
    </button>
  )
}
```

Если родитель не передал `label`, будет использовано значение `'Button'`.

```
javascript
<ActionButton onClick={handleClick} />
```

На кнопке появится:

```
text
Button
```

15. `children`: специальный проп для вложенного JSX

Иногда нужно передать не просто строку, а целый кусок JSX.

Пример:

```
javascript
function InfoBox({ children }) {
  return (
    <div className="info-box">
      {children}
    </div>
  )
}
```

Использование:

```
javascript
<InfoBox>
  <p>Props помогают передавать данные между компонентами.</p>
</InfoBox>
```

Все, что находится между `<InfoBox>` и `</InfoBox>`, попадет в проп `children`.

На первых этапах достаточно знать: `children` нужен, когда компонент должен быть оберткой для другого JSX.

16. Практический пример

Сейчас в проекте есть компонент:

```
javascript
function PageTitle() {
  return (
    <div>
      <h1>Get started</h1>
      <p>
        Edit <code>src/App.jsx</code> and save to test <code>HMR</code>
      </p>
    </div>
  )
}
```

Сделаем его гибким:

```
javascript
function PageTitle({ title, fileName, toolName }) {
  return (
    <div>
      <h1>{title}</h1>
      <p>
        Edit <code>{fileName}</code> and save to test <code>{toolName}</code>
      </p>
    </div>
  )
}
```

Теперь App передает данные:

```
javascript
<PageTitle
  title="Get started"
  fileName="src/App.jsx"
  toolName="HMR"
/>
```


Компонент `PageTitle` больше не хранит конкретный текст внутри себя. Он получает текст от родителя.

17. Практический пример: общий компонент кнопки

В третьей методичке у нас появились две вторичные кнопки:

```
javascript
<button type="button" className="secondary-button" onClick={handleResetClick}>
  Reset
</button>

<button type="button" className="secondary-button" onClick={handleHintClick}>
  Toggle hint
</button>
```

Они почти одинаковые. Отличаются только текстом и обработчиком.

Создадим компонент:

```
javascript
function ActionButton({ label, onClick }) {
  return (
    <button type="button" className="secondary-button" onClick={onClick}>
      {label}
    </button>
  )
}
```

Использование:

```
javascript
<ActionButton label="Reset" onClick={handleResetClick} />
<ActionButton label="Toggle hint" onClick={handleHintClick} />
```

Так мы убираем повторение и показываем, как props делают компонент универсальным.

18. Частые ошибки новичков

Ошибка 1. Забыть фигурные скобки при чтении props

Неправильно:

```
javascript
function PageTitle(title) {
  return <h1>{title}</h1>
}
```

В этом случае `title` будет не строкой, а целым объектом props.

Правильно:

```
javascript
function PageTitle({ title }) {
  return <h1>{title}</h1>
}
```

Или так:

```
javascript
function PageTitle(props) {
  return <h1>{props.title}</h1>
}
```

Ошибка 2. Передать число как строку

Неправильно:

```
javascript
  <Avatar size="100" />
```

Правильно:

```
javascript
  <Avatar size={100} />
```

Ошибка 3. Вызвать функцию вместо передачи

Неправильно:

```
javascript
  <ActionButton label="Reset" onClick={handleResetClick()} />
```

Правильно:

```
javascript
  <ActionButton label="Reset" onClick={handleResetClick} />
```

Ошибка 4. Попытаться изменить props внутри ребенка

Неправильно:

```
javascript
function CounterButton({ count }) {
  count = count + 1
  return <button>{count}</button>
}
```

Правильно:

```
javascript
function CounterButton({ count, onClick }) {
  return <button onClick={onClick}>{count}</button>
}
```

Изменение состояния остается в родителе.

19. Практика

Задание 1

Переделайте `PageTitle`, чтобы он принимал `props`:

```
javascript
function PageTitle({ title, fileName, toolName }) {
  return (
    <div>
      <h1>{title}</h1>
      <p>
        Edit <code>{fileName}</code> and save to test <code>{toolName}</code>
      </p>
    </div>
  )
}
```

Задание 2

Используйте `PageTitle` так:

```
javascript
<PageTitle
  title="Get started"
  fileName="src/App.jsx"
  toolName="HMR"
/>
```

Задание 3

Создайте компонент `ActionButton`:

```
javascript
function ActionButton({ label, onClick }) {
  return (
    <button type="button" className="secondary-button" onClick={onClick}>
      {label}
    </button>
  )
}
```

Задание 4

Замените обычные вторичные кнопки на:

```
javascript
<ActionButton label="Reset" onClick={handleResetClick} />
<ActionButton label="Toggle hint" onClick={handleHintClick} />
```

Задание 5

Ответьте письменно:

- какие `props` получает `PageTitle`;
- какие `props` получает `ActionButton`;
- почему `CounterButton` не должен сам менять `count`.

20. Контрольные вопросы

1. Что такое props?
2. Кто передает props?
3. Кто получает props?
4. В каком направлении обычно идут данные в React?
5. Как передать строку через props?
6. Как передать число через props?
7. Как передать функцию через props?
8. Как прочитать props через объект props?
9. Как прочитать props через деструктуризацию?
10. Почему props нельзя изменять внутри дочернего компонента?
11. Чем props отличаются от state?
12. Почему CounterButton получает count, но не хранит его сам?
13. Что такое children?
14. Зачем нужны значения props по умолчанию?
15. Как props помогают переиспользовать компоненты?

21. Краткий итог

Props - это данные, которые родительский компонент передает дочернему компоненту. Они похожи на аргументы функции и позволяют делать компоненты гибкими.

Главная схема:

text

App передает props

↓

Дочерний компонент получает props

↓

Дочерний компонент отображает интерфейс на основе props

Главные правила:

text

Props передаются сверху вниз.

Props нельзя изменять внутри дочернего компонента.

Если данные должны меняться, используем state.

Функции тоже можно передавать через props.